

Go-Live Brief

Everything standing between HOAhx today and real communities using it, in two parts. **Part One is what you decide and approve. Part Two is a checklist for your developer**, written so it can be picked up and coded without reading anything else.

Realistic time to launch ~3 weeks Findings verified against the live codebase on 21 Aug 2026

9 Shipped features that fail the moment a resident uses them	19 Policy decisions only the owners can make	0 Manual regression checks ever executed	\$0 Real money moved end to end	21 Developer tasks, ordered and estimated
--	--	--	---	---

PART ONE · FOR THE OWNERS

Seven gates between here and live

Each gate states a real problem in plain terms, what you get when it closes, and how you know it's actually done. They're ordered because each depends on the last — you cannot certify a build that isn't deployed, and you shouldn't open the doors on a build no human has walked through.

01 Fix what's broken, then put the real build in production

4–6 days

Two separate problems get solved together here. First: **nine features that residents will use on day one currently fail**. Anywhere the product asks someone to attach a file — a photo on a maintenance request, evidence on a violation report, a governing document, meeting minutes, a community feed image, an architectural review submission, a directory contact, a household photo, a profile picture — the upload is blocked by the platform's own security rules. Attaching a photo to a maintenance request is one of the two workflows the product is sold on.

Second: **the hardened, tested version of HOAhx has never actually run in production**. The live environment is still the older, pre-recovery build. Everything later on this page has to be certified against the build that will really launch, not a stand-in — so this comes first.

Two smaller items ride along: several internal reports are currently published on the public website where anyone can read them, and the software libraries the product is built on have known security holes, including nine libraries that are installed but never used at all.

DONE WHEN

All nine attachment flows work in a real browser against the live security rules; a full click-through of every page produces zero database errors; the internal reports return "not found" on the public site; and no critical or high security alerts remain in the dependency scan.

02

Watch a real dollar move

2–4 days

Dues, autopay, refunds, and subscription billing all work in a controlled test environment.

No real money has ever travelled the full path. The final acceptance step against Stripe's live checkout has not been run.

Until it is, the first resident who pays their dues is the experiment. This gate makes that first payment a routine event instead.

DONE WHEN

A live payment, a live autopay run, a live refund, and a live dispute have each completed on Stripe's real rails and been confirmed to land correctly in the community's ledger.

03

Close the doors you don't know are open

2–3 days

The day the web address is public, anyone on the internet can call your system directly rather than through the website. **The good news: they still can't get into a community they don't belong to.** That protection is real and well built.

What's missing is everything around it. Nothing limits how many times someone can try — so an attacker can guess community join codes until one works, hammer the login page, or simply run your cloud bill up with no ceiling. There is no spending cap and no budget alert today. There are also no standard browser security protections on the website, which is the first thing any corporate customer's security scan will check.

DONE WHEN

Automated abuse protection is switched on and enforcing; a spending cap and budget alert reach a real person; join-code guessing is rate-limited; and a public security-header scan of the site passes.

04

Have a human walk the whole product

2–4 days

The written regression catalog covering every product area and all 14 roles is one of the best assets this project has, and **none of it has ever been run.** Automated tests pass, but no person has logged in as a treasurer, a tenant, a vendor and a property manager and confirmed the product does what it claims.

The broken uploads in Gate 1 are the argument for this gate: they were sitting in shipped code, and no automated test caught them.

DONE WHEN

A prioritized slice of the catalog has been executed against the exact build that will launch, with results recorded, and someone has signed a written go / no-go recommendation.

DEVELOPER TASKS **D16**

05 Lock the accounts that control money

2–3 days

A treasurer can issue a refund of any size with no second signature. A president can place a lien on someone's home. Platform staff can view every community. **All of these accounts are protected by a six-character password and nothing else** — no second factor, no breach checking.

HOA boards are a standard phishing target precisely because members are volunteers, often non-technical, and usually listed publicly on the community website.

DONE WHEN

App-based two-factor sign-in is required for platform staff and board roles, the password floor is raised with breach screening on, and refunds require a fresh password entry. Residents stay on an opt-in path so homeowners aren't disrupted at launch.

DEVELOPER TASKS **D11**

06 Be able to see trouble, and to undo it

2–3 days

Today there is **no early-warning system of any kind**. If payments start failing at two in the morning, your first alert is an angry board member the next day. The broken uploads prove the point — a whole category of features could fail silently in production with nothing to tell you.

There is also **no backup**. You are holding other people's financial ledgers, governance records, and legal documents, and the system contains bulk operations powerful enough to do real damage by accident. One mistaken action is currently unrecoverable.

DONE WHEN

A deliberately triggered test error reaches a human within minutes; daily backups and point-in-time recovery are switched on; and one full restore has actually been rehearsed and written down.

DEVELOPER TASKS **D12** **D13**

07 Decide the policies the software can't decide for you

1–2 days

A number of the product's rules were never actually decided — the code simply does something, and that something became the policy by default. A treasurer can refund any amount alone because nobody set a threshold. Late fees never get charged because nobody defined when. A lien needs one click because nobody required a vote.

These are business decisions, not engineering ones, and several carry legal or financial exposure. The nineteen that genuinely gate launch are listed below, each with what the software does today, so answering is a yes-or-no rather than a research project.

DONE WHEN

All nineteen decisions are answered and written down, and any that change behavior are handed to the developer as scoped work.

One thing to correct before you sell it

Mass email doesn't send. "Communications" is positioned as a launch feature, and the in-app message centre works — but the email and SMS channels record themselves as *not configured*. A board that sends a community-wide notice today reaches only residents who happen to open the app. Wiring email is a first-30-days item (**D14**), not a launch blocker — but the marketing claim needs to match reality until it's done.

The nineteen decisions that gate launch

Each is listed with what the software does *today*. Where the answer changes behavior, it becomes a scoped task for the developer — but most simply need to be decided and recorded.

MONEY — YOU CANNOT SELL OR COLLECT WITHOUT THESE

What are the actual plans and what do they cost?

TODAY The billing machinery works, but no plan catalogue is fixed. There is nothing to sell yet.

What happens to a community when its subscription lapses?

TODAY Nothing. The lapse is recorded but never restricts access, so an unpaid community keeps full use of the platform indefinitely.

Is there a free trial, and what happens when it ends?

TODAY The system understands a trial state, but nothing ever starts or ends one.

Should a refund above a set amount need a second approver?

TODAY No threshold and no co-approval. A single treasurer can refund any payment, at any amount, alone.

Are late fees applied automatically, and on what schedule?

TODAY Late-fee settings are stored, but nothing ever assesses one. Late fees exist on paper only.

When an autopay attempt fails, what happens next — retry, notice, escalation?

TODAY The failure is recorded. There is no retry schedule and no collections policy, so that money is simply not collected.

Who gets told when a resident disputes a payment?

TODAY Nobody. The ledger adjusts silently — the board finds out by noticing the number changed.

LEGAL EXPOSURE — THESE PUT LIENS ON PEOPLE'S HOMES

Must a waiting period pass between a violation notice and a fee?

TODAY No waiting period is enforced anywhere. A fee can follow a notice immediately.

Is a resident guaranteed a hearing before a lien is placed?

TODAY No. There is no appeal step in the path from delinquency to lien.

Should escalating to a lien require a recorded board vote?

TODAY One president acting alone, with no conflict-of-interest check even when the president is the subject.

Is the built-in signature legally sufficient for the documents being signed?

TODAY A native acknowledgement only — no third-party e-signature provider. This needs a lawyer's answer, not a developer's.

PRIVACY AND RECORDS — THE FIRST QUESTION ANY CUSTOMER ASKS

How long is each kind of record kept?

TODAY No retention policy exists anywhere. Everything is kept forever, including violations after they're cured.

If a resident asks for their data, or asks you to delete it, what happens?

TODAY A resident can delete their own account. There is no export path, and what happens to their records afterward is undefined.

Can audit records be exported for a legal or regulatory request?

TODAY No export path exists. The records are kept but can't be produced.

Are there notices a resident may *not* switch off — emergency, legal, or dues?

TODAY No mandatory category exists. A resident can silence a legal notice, and the system will honor it.

ACCESS AND OPERATIONS

Two-factor sign-in for staff and board roles, and how strong should passwords be?

TODAY No two-factor anywhere, and a six-character password floor with no breach checking. This is what Gate 5 acts on — your answer sets its scope.

When a management company's contract with a community ends, what happens to their access and their data?

TODAY There is no offboarding path at all. You will hit this with your first management-company customer.

Is archiving a community reversible, and how long is archived data kept?

TODAY Archiving is final. There is no stated retention period and no restore path.

Should a community be blocked from going live until its setup is actually complete?

TODAY Setup readiness is measured and displayed, but never enforced. A half-configured community can open its doors.

After you're live

Launch is a day; keeping the platform trustworthy is the job. Three things run in parallel once the doors open.

THE STANDING RHYTHM

EACH RELEASE

Release-gate rerun. The cross-role test campaign runs against every candidate before it ships, so nothing quietly undoes the recovery work.

MONTHLY

Security and dependency patching. Libraries and runtimes kept current, before an update becomes an emergency.

MONTHLY

Payments and platform health review. Billing events, failure rates, error trends, and infrastructure spend, written up in a page you can hand to a board.

QUARTERLY

Access and permissions audit. Role grants, temporary vendor and auditor access, and impersonation logs reviewed so authority never quietly accumulates.

THE FIRST 30 DAYS

WORK	WHY IT MATTERS COMMERCIALY	TASK
Wire mass email, and move sending to a queue	You can't sell "Communications" until a board-wide notice actually reaches inboxes. A 1,000-home community also currently exceeds the send limit halfway through.	D14
Require a verified email before joining a community	Fine while access is invite-led. It stops being fine the moment self-serve signup drives growth.	D15
Make the automated safety net trustworthy	The strongest tests never run automatically, and a failing test currently blocks nothing. Fixing this is what stops a repeat of the broken uploads.	D16
Cover the background jobs	21 background processes have effectively no behavioral tests, and when one fails it fails <i>silently</i> — a record simply never gets written.	D17

WORK	WHAT GROWTH DOES TO IT	TASK
Speed up first page load	Every visitor downloads the entire platform — all four portals — just to see the login screen. Multi-second waits on the mid-range phones residents actually use, against a "mobile-first" promise.	D18
Page the big lists	Admin screens load every record every time. A 1,500-home community turns one roster view into thousands of billed database reads.	D19
Split the backend	All 197 backend functions deploy together, all or nothing, taking 15+ minutes and failing as a unit.	D20
Fix multi-community sessions	A management-company admin working on a laptop in one community silently breaks their own phone session in another. Your highest-value users hit this daily.	D21

What's already solid — and it's the expensive part

Everything above sits on a genuinely sound foundation, which is worth stating plainly. Per-community data isolation is enforced by server-issued credentials, not by hiding buttons. Every backend mutation re-checks authority against the token. Stripe uses a signature-verified webhook with a duplicate-event ledger and server-owned payment state. Invite tokens are 144-bit random, sensitive operations are idempotent, email templates escape user input, impersonation is time-limited and fully audited, and the hardcoded admin backdoor is verifiably gone. **The hard, expensive part is done. What remains is the last mile.**

Twenty-one tasks, in order

Every task names the evidence in the codebase, the fix, and the test that closes it. All findings were verified against the working tree on 21 August 2026. Tick the boxes as you work through them.

Before the doors open Blocks launch · ~6–8 days

☐ D1 **CRITICAL** BROKEN FEATURE

2–3 days

Nine upload features write to a storage path the rules deny outright

EVIDENCE

`storage.rules:324` declares `match /uploads/{fileName} { allow read, write: if false; }`, while `src/api/firestoreAdapter.ts:719` still uploads to `uploads/${Date.now()}_${file.name}`. Confirmed call sites:

- `components/maintenance/SubmitTicketModal.tsx`
- `components/violations/SubmitViolationReportModal.tsx`
- `components/documents/UploadDocumentModal.tsx`
- `components/voting/MinutesUploadModal.tsx`
- `components/directory/ContactFormModal.tsx`
- `components/profile/PhotoUpload.tsx` — calls the generic `UploadFile`, not the working `UploadProfileImage`
- `pages/MyHousehold.tsx`
- `pages/architectural/ArchitecturalReviewPage.tsx`
- `pages/feed/CommunityFeed.tsx`

FIX

Route each call site to a scoped, rule-governed path. **The pet-photo and message-attachment paths are already correct — use them as the template.** Add matching `storage.rules` blocks with content-type and size limits per surface. Point `PhotoUpload.tsx` at `UploadProfileImage`.

DONE WHEN

All nine flows complete in a browser against deployed rules, and a UI test covers at least the maintenance and violation photo paths.

☐ D2 **CRITICAL** DEPLOYMENT

1 day + scripting

Production runs the pre-recovery build, and the deploy path is manual and order-fragile

EVIDENCE

The verified recovery batches are on `dev` and not deployed. Deployment requires a strict manual order — indexes → functions → frontend → rules — documented in `docs/PHASE3_RULES_REMEDIATION.md`, and the rules file itself warns that deploying out of order locks existing users out.

FIX

Deploy in order on staging first, then production. Then collapse the four steps into one script and add it to CI as a gated deploy job, so the order can never be violated by hand under launch pressure.

DONE WHEN

Production serves the `dev` build, sessions created before the claims migration still work, and the deploy is a single command.

☐ D3 **HIGH** RELIABILITY

1 day

Missing composite indexes will hard-fail queries at runtime

EVIDENCE

Property-scoped and dog-park composite indexes are still marked "to be captured on staging," meaning some sorted and filtered query shapes in `firestore.indexes.json` are known-incomplete. Firestore raises `failed-precondition` the first time a real user hits an unindexed shape — it surfaces as a broken page, never as a build failure.

FIX

Exercise every route on staging with seeded data, capture the index-creation links Firestore emits, commit them to `firestore.indexes.json`, and deploy indexes first in the D2 ordering.

DONE WHEN

A full staging click-through of all 124 routes produces zero `failed-precondition` errors.

☐ D4 **HIGH** DATA EXPOSURE

1 hour

Internal audit reports ship into the public production build

EVIDENCE

`vite.config.js:11-18` lists `client-walkthrough.html`, `common-workflow-test-run.html`, `manual-ui-tests.html`, `project-impact-report.html`, and `roles-permissions-report.html` as rollup inputs. They land in `dist/` and Vercel serves them at guessable URLs — exposing the complete role and permission architecture, internal project statistics, and the full test catalog.

FIX

Gate the extra inputs behind an env flag that defaults off for production builds, or move the documents behind authentication.

DONE WHEN

A production build contains only `index.html`, and the five URLs return 404 on the live site.

☐ D5 **HIGH** SUPPLY CHAIN

0.5 day

Known-vulnerable dependencies, nine of which are never imported

EVIDENCE

`npm audit`: 13 frontend vulnerabilities (2 critical, 4 high) and 14 in `functions/` (1 critical). Verified as present in `package.json` with **zero imports anywhere in `src/`**: `react-quill` (drags in Quill ≤1.3.7 XSS), `three`, `moment`, `lodash`, `react-leaflet`, `html2canvas`, `canvas-confetti`, `@hello-pangea/dnd`, `react-markdown`. `jspdf` has one real import and a live object-injection advisory at ≤4.2.0.

FIX

Uninstall the nine unused packages, run `npm audit fix` in both trees, and upgrade `jspdf` past the advisory.

DONE WHEN

`npm audit` reports no critical or high findings in either tree, and the build still passes.

☐ D6 **HIGH** RELIABILITY

5 min

firebase-functions is pinned to **"latest"**

EVIDENCE

`functions/package.json:17` declares `"firebase-functions": "latest"`. The lockfile pins it today, but any `npm install` or lockfile refresh silently jumps a major version — potentially minutes before a deploy, taking all 197 functions down.

FIX

Pin to a caret range at the currently-locked major and let dependency PRs handle upgrades deliberately.

DONE WHEN

No production dependency in either tree resolves to `latest` or `*`.



D7 **CRITICAL** PAYMENTS

2–4 days

Live Stripe acceptance has never been executed

EVIDENCE

Hosted Stripe Checkout has only ever run against emulators and signed test webhooks. The full procedure already exists at `docs/STRIPE_TUESDAY_RUNBOOK.md` and has not been run.

FIX

Run the runbook end to end: production preflight, versioned 15-event webhook endpoint, bind the intended secret versions, redeploy functions, then execute the hosted flows.

DONE WHEN

A live one-time payment, an autopay run, a refund, and a dispute have each completed and reconciled correctly against the property ledger.

Launch week Before traffic arrives · ~5 days



D8 **HIGH** ABUSE & COST

2 days

172 public callables with no App Check, no rate limiting, and no instance caps

EVIDENCE

Every callable uses `cors: true, invoker: 'public'`. Verified: **zero occurrences** of `appCheck`, `maxInstances`, or `setGlobalOptions` anywhere in `src/` or `functions/`. The only rate limiter in 16,592 lines guards platform recovery at `functions/index.js:16494`. Authorization holds — membership is checked — but nothing bounds volume or spend.

FIX

Enable App Check (reCAPTCHA Enterprise) in enforcement mode; add `setGlobalOptions({ maxInstances })` with per-function overrides on hot paths; rate-limit the email-sending and auth-adjacent callables using the existing recovery limiter as the pattern; set GCP budget alerts today — they're free and take minutes.

DONE WHEN

App Check is enforcing, a synthetic call without a valid token is rejected, and a budget alert fires to a human at a test threshold.



D9 **HIGH** SECURITY

0.5 day

HOA join codes can be brute-forced into real memberships

EVIDENCE

`redeemAccessCode` has no attempt limiting. Access codes are admin-supplied strings with no enforced length or entropy — `OAKW00D2026` is a legal code — and codes with `auto_approve` grant membership instantly. Generated codes fold `base64url` through `.toUpperCase()` at `functions/index.js:7140`, cutting per-character entropy; the 6-character clubhouse codes land near 31 bits.

FIX

Rate-limit redemption per user, enforce a minimum length and entropy at code creation, alert on repeated failures, and generate from an explicit unambiguous alphabet instead of uppercasing `base64url`.

DONE WHEN

A scripted guessing run is throttled and alerts, and newly generated codes clear the entropy floor.



D10 **MEDIUM** SECURITY

0.5 day

No security headers on the production host

EVIDENCE

Verified: `vercel.json` contains only the SPA rewrite — no Content-Security-Policy, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, or Permissions-Policy on any response.

FIX

Add a `headers` block: CSP allowing the Firebase and Stripe origins, `frame-ancestors 'none'`, HSTS, nosniff, and a restrictive Permissions-Policy.

DONE WHEN

A public header scan of the production domain passes, and the app still loads Firebase and Stripe without CSP violations in the console.



D11 **HIGH** IDENTITY

2–3 days

Six-character passwords and no MFA for roles that move money

EVIDENCE

`src/pages/auth/Register.tsx:20` surfaces Firebase's 6-character default. There is no `multiFactor` usage anywhere in `src/`. Treasurers, presidents, and the platform owner authenticate with exactly the same floor as residents. A recommendation exists at `docs/AUTHENTICATION_MFA_RECOMMENDATION.md` and was never implemented.

FIX

Turn on the Identity Platform password policy with length plus breach screening; offer TOTP and require enrollment for platform and board roles; require recent re-authentication before refunds and payout-adjacent actions.

DONE WHEN

A platform or board role cannot complete sign-in without a second factor, and a refund attempt on a stale session prompts for re-authentication.

First 30 days While traffic is still forgiving



D12 **HIGH** OPERATIONS

1–2 days

No error monitoring or alerting anywhere in the stack

EVIDENCE

Verified: no Sentry, LogRocket, or Crashlytics in the client and no alerting configuration for functions. Errors reach `console.error` and stop there. No uptime checks and no Stripe-webhook-failure alerts. D1 is the proof case — nine features broken in shipped code with nothing to report it.

FIX

Client error reporting (a free Sentry tier is sufficient), Cloud Error Reporting alerts on function error rate, uptime checks on the app and the Stripe webhook endpoint, and a paging channel that reaches a human.

DONE WHEN

An intentionally thrown test error pages a person within minutes.



D13 **HIGH** RECOVERY

1 day

No backup or point-in-time recovery posture for Firestore

EVIDENCE

No scheduled exports and no PITR configuration anywhere in the repo or docs, while the codebase contains legitimately destructive batch operations — `revokeAllStandingAccess` strips every grant a user holds.

FIX

Enable Firestore PITR with a 7-day window, schedule daily exports to a GCS bucket with retention, and run one restore drill so the procedure is written down before it is needed.

DONE WHEN

One restore has been completed from an export into a scratch project, and the steps are documented.



D14 **MEDIUM** SCALE & CORRECTNESS

2–3 days

Mass communication fans out inline, and the email channel is not wired

EVIDENCE

`sendMassCommunication` (`functions/index.js:15600+`) resolves recipients then writes per-recipient × per-channel delivery receipts synchronously inside the request. A 1,000-resident community across two channels is 2,000+ writes against a 60-second callable ceiling — partial delivery with no resume. Email and SMS channels record `not_configured`; in-app is the only channel that delivers.

FIX

Enqueue the send via Cloud Tasks and deliver in idempotent chunks from a worker — the receipt IDs are already deterministic, so this is straightforward. Wire Resend for the email channel before mass email is positioned as shipped.

DONE WHEN

A mass communication to a 1,000-recipient seed completes fully, and a real email arrives at a test address.



D15 MEDIUM TRUST MODEL

0.5 day

Unverified email addresses keep full session access

EVIDENCE

Registration sends a verification email, but nothing gates the app on it — `emailVerified` from `AuthContext` is consumed only by the invite flow. Firestore rules do enforce verified email where identity is critical (messaging participants, email-bound records), which is the right server-side instinct.

FIX

Gate access-code redemption and HOA access requests on a verified email; add a soft verification banner elsewhere. Tolerable while access is invite-led; do it before opening self-serve growth.

DONE WHEN

An unverified account cannot redeem an access code or file an access request.



D16 HIGH TEST PIPELINE

1 week

The strongest tests never run, and a failing test blocks nothing

EVIDENCE

Verified: `.github/workflows/ci.yml` runs build, unit, rules, lint and typecheck — **and has no Playwright step at all**, so the 30 browser specs under `tests/ui/` covering 15 roles never execute. The toolchain is also unreproducible: `firebase-tools` is not declared as a dependency in either tree, and `.nvmrc` says 20 while `functions/package.json` requires Node 22 — so local runs execute Cloud Functions on a runtime production never uses.

FIX

- Pin the toolchain: `firebase-tools` as a fixed devDependency, `.nvmrc` to 22, document the Java 21 requirement, drop CI's global unpinned install.
- Wire a prioritized tier of the UI suite into CI on every PR, and run the full suite nightly — the whole suite is too slow to gate every push.
- **Enable branch protection on main and dev**. CI currently reports and never gates — a red run blocks nothing today. This is a repository setting the owner must make.
- Separate the unit tests that assert on source text from those that execute behavior, so the coverage number means something. Don't delete them — they encode real invariants.

DONE WHEN

A pull request with a deliberately broken behavior is blocked by CI.



D17 HIGH TEST COVERAGE

3–4 days

Effectively zero of 21 Firestore triggers have behavioral coverage

EVIDENCE

18 triggers are referenced by no test at all; the three that are referenced appear only in tests that grep source text. That includes claims re-issue on membership-role and property-role writes, every audit projection, announcement fan-out, and idempotency under platform retry. Trigger failures are **silent** — a projection simply never gets written and nothing errors.

FIX

Emulator-backed tests per trigger asserting the resulting database state, plus a double-delivery case for idempotency. Add an export-coverage sweep as a CI check that fails when a new `exports.*` lands with no test reference.

DONE WHEN

Every trigger has an executing test, and the export-coverage check is green in CI.

□ D18 **HIGH** PERFORMANCE

1 week

One 3.9 MB JavaScript bundle, zero code splitting

EVIDENCE

The built bundle is 3.9 MB (~1.0 MB gzipped). Verified: **zero occurrences of `React.lazy`** across 52 page modules — every visitor downloads all four portals, Recharts, Framer Motion and Firebase to see the login screen. Every deploy also invalidates the whole bundle for every user.

FIX

Route-level `React.lazy` per portal — resident, admin, platform, management are natural seams — plus a `manualChunks` vendor split.

DONE WHEN

The initial route is under 300 KB gzipped.

□ D19 **MEDIUM** COST & LATENCY

3–5 days

Fetch-everything data access with client-side filtering and no pagination

EVIDENCE

The adapter's `list()` and `filter()` default to unbounded reads. `getHoaRoster` scans five collections per call — memberships, roles, property roles, households, legacy users — plus batched `in` queries, per admin page view, with no cache and no pagination. The `users` path queries broad and filters in the client.

FIX

Default limits and cursor pagination in the adapter; paginate the roster callable; maintain a per-HOA roster aggregate updated by the existing membership triggers, so list views read one document instead of five collections.

DONE WHEN

Roster load is $O(\text{page})$ rather than $O(\text{community})$ in read count.

□ D20 **MEDIUM** DEPLOY

3–5 days

A 16,600-line functions monolith: 197 exports in one module

EVIDENCE

All functions live in one `index.js` (708 KB) plus siblings. Every instance loads the entire module graph regardless of which function it serves; deploys are all-or-nothing across 197 functions, take 15+ minutes, and brush against Cloud Functions deploy quotas.

FIX

Split into domain codebases via `firebase.json`'s multi-codebase support — identity, finance, comms, operations — so deploys and cold starts scope to a domain. The existing service-module extraction (`signupService`, `onboardingService`) shows the seams already exist.

DONE WHEN

A single-domain deploy completes in minutes and touches only that domain's functions.



D21 MEDIUM ARCHITECTURE

3–5 days

Active-context-in-claims serializes multi-community usage

EVIDENCE

`computeUserClaims` writes only the committed HOA context into the token. Every context switch is a Firestore write plus `setCustomUserClaims` plus a forced token refresh — and because claims are global per user, two devices or tabs in different communities clobber each other's access.

FIX

Carry *all* memberships in the token — the truncation fallback for the 1 KB cap already exists — and treat active context as request data validated server-side, which the callables' envelope-versus-claims check already largely does. At minimum, detect the clobber client-side and auto-recommit rather than erroring.

DONE WHEN

Two devices hold different community contexts simultaneously without permission errors.

Compiled 21 August 2026. Every technical claim in Part Two was checked against the working tree on the same day. Not covered here: live production configuration that can only be read from the consoles — secret version binding, App Check state, and the Stripe dashboard.